

Experiment 3

Aim - To Perform various GIT operations on local and Remote repositories

Theory:

The command ``mkdir git`` creates a new directory (folder) named "git" in the current working directory. This command is used to make a new directory in a Unix-like operating system.

The command ``cd git`` is used to change the current working directory to the directory named "git." After executing this command, any subsequent commands or file operations will occur within the "git" directory. "cd" stands for "change directory."

The ``git config--global user.name`` and ``git config--global user.email`` commands are used to set your global Git username and email address, respectively. They are part of the configuration settings in Git and are associated with the commits you make.

If you want to check your configuration settings, you can use the `git config--list` command to list all the settings Git can find at that point

```

Last login: Wed Apr 15 22:34:22 on console
/dev/fd/12:25: command not found: compdef
(base) mohitrajpal@Mohits-MacBook-Air ~ % git version
git version 2.51.0
(base) mohitrajpal@Mohits-MacBook-Air ~ % git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
           [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
           [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
           <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
  bisect    Use binary search to find the commit that introduced a bug
  diff      Show changes between commits, commit and working tree, etc
  grep      Print lines matching a pattern
  log       Show commit logs
  show      Show various types of objects
  status    Show the working tree status

grow, mark and tweak your common history
  backfill  Download missing objects in a partial clone
  branch    List, create, or delete branches
  commit    Record changes to the repository
  merge     Join two or more development histories together
  rebase    Reapply commits on top of another base tip
  reset     Reset current HEAD to the specified state
  switch    Switch branches
  tag       Create, list, delete or verify a tag object signed with GPG

collaborate (see also: git help workflows)
  fetch     Download objects and refs from another repository
  pull      Fetch from and integrate with another repository or a local branch
  push      Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.
(base) mohitrajpal@Mohits-MacBook-Air ~ % mkdir t21/93
mkdir: t21: No such file or directory
(base) mohitrajpal@Mohits-MacBook-Air ~ % mkdir t21-93
(base) mohitrajpal@Mohits-MacBook-Air ~ % cd desktop
(base) mohitrajpal@Mohits-MacBook-Air desktop % mkdir t21-93
(base) mohitrajpal@Mohits-MacBook-Air desktop % cd t21-93
(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git config --global --list
(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git config --global user.email "mohitrajpal283@gmail.com"
(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git config --global --list
user.name=Mohit
user.email=mohitrajpal283@gmail.com

(base) mohitrajpal@Mohits-MacBook-Air t21-93 % git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /Users/mohitrajpal/Desktop/t21-93/.git/
(base) mohitrajpal@Mohits-MacBook-Air t21-93 %

```

git commit-am "commit message" stages and commits all changes in tracked files with a commit message in a single command.

The command ``nano index.html`` opens the Nano text editor for the file named "index.html." Nano is a simple command-line text editor that allows you to view and edit files directly in the terminal.

The command ``touch teststatus`` creates an empty file named "teststatus" in the current directory. The ``touch`` command is commonly used to update the timestamps of a file or create an empty file if it doesn't exist.

`git checkout-- teststatus`: Discards changes to the file "teststatus" in the working directory. This reverts the file to the state it has in the last commit.

The ``git add`` command is used to stage changes in the working directory for the next commit in Git. It prepares modifications, additions, or deletions to be included in the upcoming commit.

The ``git log`` command is used to display the commit history of a Git repository. It shows a chronological list of commits, including commit hashes, author information, timestamps, and commit messages.

The command ``git log--oneline`` displays a simplified and concise one-line representation of the commit history in a Git repository, showing only the commit SHA-1 hash and the commit message.

The ``git clone`` command is used to create a copy of a Git repository. When you run this command, it duplicates the entire repository, including its files, commit history, and branches, and downloads it to your local machine. This is often the initial step when you want to work with a project hosted on a remote Git repository.

The ``git pull`` command is used to fetch and integrate changes from a remote repository into the current branch of your local repository. It combines two actions: it fetches the changes from the remote repository, and then it automatically merges those changes into your local branch. This is a convenient way to update your local repository with the latest changes from the remote repository.

The ``git push`` command is used to upload or push the local changes in your Git repository to remote repository. It updates the remote repository with the latest changes made in your local branch, making them accessible to others who share the same remote repository.

The ``git fetch`` command is used to retrieve changes from a remote repository. It fetches any new branches or changes made in the remote repository since your last interaction.

However, it does not automatically merge these changes into your local branches. After using ``git fetch``, you can inspect the changes and decide whether to integrate them using ``git merge`` or ``git rebase``.